OUTBOX RELIABILITY LAB

From job-market research to a verified backend engineering MVP

Prepared for: Ahmad Essam Abdelaziz **Date:** 1 November 2026 **Delivery:** local MVP + source archive

A simulated order system that survives temporary processing failures and demonstrates why idempotency matters. Built with Python, FastAPI, SQLite, a separate polling worker, tests, and a read-only dashboard.

1 | Why this project won

Current employer postings emphasize reliable APIs, asynchronous processing, testing, debugging, databases and operational ownership. Ahmad already has AI evaluation and full-stack experience; a demonstrable failure-tolerant backend closes a different gap.

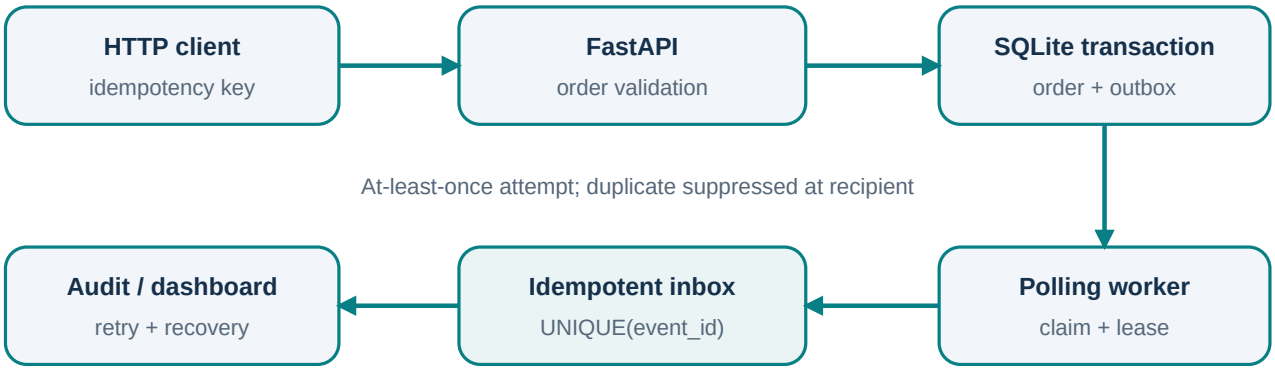
Ranked idea	Hiring / gap / feasibility	Editorial score
1. Outbox Reliability Lab	Reliability, transactional writes, retries, auditability	97/100
2. ReconcileAudit	Finance automation and matching	85/100
3. SearchRankBench	Search relevance and evaluation	82/100
4. EdgeVisionOps	Computer vision inference and MLOps	75/100

Scoring rubric: hiring demand 25%, gap 20%, usefulness 15%, differentiation 15%, feasibility 15%, low cost 10%. Scores are editorial assessments, not hiring probabilities.

Hiring evidence

Monzo: Go, Kafka, Docker and shipped code. **Amazon:** reliable services, CI/CD and code verification. **Cloudflare:** Go/Python and large-scale engineering. **Graphcore:** Linux, monitoring and automation. **Aleph Cairo:** process automation and reconciliation. These are real employer descriptions, not guarantees of eligibility.

2 | Architecture and data flow



All components run locally. Inbox is simulated, not a real external service.

What happens step by step

1. The API checks an integer amount and the Idempotency-Key header. It rejects invalid requests and conflicting repeated keys.

2. A single SQLite transaction inserts the order and an outbox event. If the outbox insert fails, the order rolls back.

3. A separate worker claims the oldest due event using BEGIN IMMEDIATE, records an attempt and a time-limited lease.

4. The worker writes the event to a simulated downstream inbox with UNIQUE(event_id). The same event can be retried without a second effect.

5. Successful delivery is acknowledged. Temporary failures are retried with bounded backoff; repeated failures are dead-lettered.

6. If a crash occurs after the inbox commit but before acknowledgement, the lease expires, recovery requeues the event and the inbox suppresses the duplicate.

Why it matters

A common bug is saving an order and then crashing before notifying another service. The transactional outbox removes that local gap. At-least-once delivery can still replay messages, so a receiver needs durable idempotency. This lab makes both issues observable and testable.

3 | Actual input-to-output walkthrough

The recorded demo ran on 1 November 2026 against a local SQLite file with synthetic amounts. Its two events followed these paths:

Event	Injected behavior	Observed result
Order A: 1,250 cents	Failure before delivery	retry, then delivered
Order B: 2,500 cents	Crash after inbox commit, before ack	lease recovered; duplicate suppressed

Recorded demo counts: 2 orders; 2 unique downstream deliveries; 10 audit entries; all events ended in delivered state.

Example HTTP request

`POST /orders` with `Idempotency-Key: sample-001` and JSON `{"amount_cents":1250}`. The response includes a generated order_id and event_id. After one worker run, `GET /orders/{id}` returns `"status":"delivered"`. A duplicate request with the same key and amount reuses the original order; a different amount returns HTTP 409.

Key source files

File	Responsibility
db.py	SQLite schema, WAL, connection setup
service.py	Atomic order/outbox, leases, retries, delivery, audit
api.py	Validated REST API and read-only dashboard
cli.py	Worker loop, failure injection and deterministic demo
tests/	Rollback, concurrency, recovery, dead-letter, HTTP flow
MASTER_PROMPT.md	Research-based, pre-code implementation contract

4 | Installation, verification and limitations

How to run locally

- `python -m venv .venv` and activate it.
- `python -m pip install -e '[dev]'`
- `outbox-lab --db demo.sqlite demo`
- `python -m pytest -q`
- `OUTBOX_DB=demo.sqlite uvicorn outbox_lab.api:app`; visit `/dashboard` and `/docs`.

Verification actually performed

Check	Observed outcome
Unit/integration/API tests	16 passed
Python compileall	Passed
Editable install	Passed with --no-build-isolation --no-deps (offline environment)
CLI failure/recovery demo	Passed; 2 orders, 2 unique deliveries
HTTP create → worker → read	HTTP 201 create, 200 read, delivered state
Docker Compose	Not run; Docker unavailable
Remote GitHub CI	Not run; repository creation blocked

Investigated blocker

Standard pip editable install initially failed because the isolated build could not resolve PyPI DNS while fetching setuptools. Installed setuptools 82.0.1 was available locally; installing without build isolation and without dependency downloads worked, followed by a successful full test rerun. Normal installation instructions remain valid on a network-enabled machine.

Important limitations

No authentication, external message broker, real payments, multi-node deployment, cloud monitoring or real external recipient. The inbox shares a SQLite file with the outbox but commits separately. A simulated crash does not kill the OS process. No Docker, GitHub Actions or external performance benchmark was verified. Do not present this as a production banking system.

5 | GitHub, interview and CV presentation

GitHub publication status

Local feature branch: `feat/outbox-reliability-mvp`. **Verified local code commit:** `864f5e2` (the explanatory PDF is generated afterward). **Requested remote:** Ahmad-Joker/outbox-reliability-lab. The repository returned 404 and no available connected action can create it; gh CLI is absent. Therefore **no remote push or pull request exists**. A recoverable source ZIP is supplied. To publish, create the dedicated repository and grant connected GitHub access; then inspect its default branch and open a feature PR.

How to explain it in an interview

“I built a small fault-injection lab for the transactional outbox pattern. A single transaction saves an order and event; a separate worker claims events, retries temporary failures and records audit history. I simulated a crash after the downstream side effect but before acknowledgement. The event was replayed, yet a unique event ID prevented a second effect. I tested rollback, concurrent workers, retry exhaustion and the HTTP flow. It is a local teaching system, not a production distributed broker.”

Truthful CV bullet

Built a Python/FastAPI and SQLite transactional-outbox lab with idempotent event consumption, leased worker recovery, bounded retries, failure injection, automated tests and an audit dashboard; verified crash-after-delivery recovery without duplicate simulated side effects.

Saved master prompt: key commitments

The pre-code MASTER_PROMPT.md records the hiring evidence, weighted ranking, target users, user stories, architecture, exact MVP boundary, modules, edge cases, acceptance criteria, implement–test–inspect–fix loop, GitHub publication steps and explanatory PDF requirements. The actual implementation followed the contract; the GitHub stage is explicitly blocked.

Employer and technical sources (official postings unless noted)

- Monzo internship
- Amazon ML SDE internship
- Cloudflare internship
- Graphcore platform internship
- Aleph Cairo internship
- Microservices.io: idempotent consumer (pattern reference)
- Temporal design patterns (competing platform)